



SIMATS
ENGINEERING



SIMATS
Saveetha Institute of Medical And Technical Sciences
(Declared as Deemed to be University under Section 3 of UGC Act 1956)

SIMATS ENGINEERING

Saveetha Institute of Medical and Technical Sciences

Design and Develop an Intelligent Flood Disaster Detection and Emergency Alert Prediction System Using Deep Learning

A COURSE ASSIGNMENT REPORT

Submitted in the partial fulfilment for the Course of

MLA0403 – Deep Learning

to the award of the degree of

BACHELOR OF ENGINEERING

IN

B TECH AI&ML

Submitted by

D N SHIRSHIKA(192425396)

K.NEEHA (192425245)

P.HITHAISHI (192425196)

Under the Supervision of

Dr.A.M.ARUL RAJ

Chennai-602105

Date of Submission – Sep 2026

Table of contents

Chapter.No	TITLE	PAGE NO.
1	Problem statement	4
2	Aim and Objective	6
3.	Requirements / Environment and Tools Used	7
4.	Design of the Module	8
5.	Proposed Solution	9
6.	Algorithm	10
7.	Pseudocode	14
8.	Flowchart	16
9.	Implementation	17
10.	Source Code	22
11.	Test Cases	
12.	Expected/Actual Results	24
13.	Execution Screenshots / Output	
14.	Analysis Result charts	25
15.	Discussion and Limitations	
16.	Conclusion	
17.	Future Enhancement	
18.	Individual Contribution of Group Members	26
	Reg.No Name Assigned Task	
19.	References	

List of Table

S.No	Table Name	TABLE NO	Page No
1	Stastitical summary	1	14
2	Individual Contribution of Group Members	2	22

List of Figures

S.No	Fig. Name	Fig NO.	Page No
1	Flood risk class Distribution	1	14
2	Correlation matrices of flood-related features	2	15
3	Training and Validation	3	16

“Design and Develop an Intelligent Flood Disaster Detection and Emergency Alert Prediction System Using Deep Learning”

1. Problem Statement

Traditional flood monitoring systems mainly depend on manual observation, weather reports, and delayed environmental analysis. These approaches may not provide sufficiently early identification of changing flood conditions and may have difficulty analyzing large amounts of environmental data. Therefore, there is a need for an intelligent system that can automatically analyze flood-related environmental information and identify potential flood conditions.

The proposed system uses Deep Learning to analyze relevant flood-related data and classify the flood-risk level into categories such as Normal, Low Risk, Moderate Risk, High Risk, and Severe Flood Warning. The system also provides an emergency alert based on the predicted risk level, helping authorities and communities take timely preventive action.

2. Objective

The main objectives of the proposed system are:

1. To develop a Deep Learning-based system for detecting and predicting flood-risk conditions.
2. To preprocess and analyze environmental parameters such as rainfall, water level, temperature, humidity, and historical flood information.
3. To design an appropriate Deep Learning model capable of learning complex patterns from flood-related data.
4. To classify the detected condition into different flood-risk levels.
5. To generate an understandable emergency alert according to the predicted flood-risk level.
6. To evaluate and improve the model using accuracy, precision, recall, F1-score, confusion matrix, and loss analysis.

3. Requirements and Environment Used

Hardware Requirements

- Computer/Laptop

- Minimum 4 GB RAM
- Intel/AMD processor
- Internet connection
- GPU support through Google Colab, if available

Software Requirements

- Python 3.x
- Google Colab / Jupyter Notebook
- TensorFlow
- Keras
- NumPy
- Pandas
- Matplotlib
- Scikit-learn
- Seaborn

Dataset Requirements

The system requires flood-related environmental data containing suitable parameters such as:

- Rainfall intensity
- Water/river level
- Temperature
- Humidity
- Historical flood records
- Location/geographical information

The assignment also permits satellite, surveillance, flood and non-flood images where an image-based approach is selected.

Development Environment

Programming Language: Python

Development Platform: Google Colab / Jupyter Notebook

Deep Learning Framework: TensorFlow and Keras

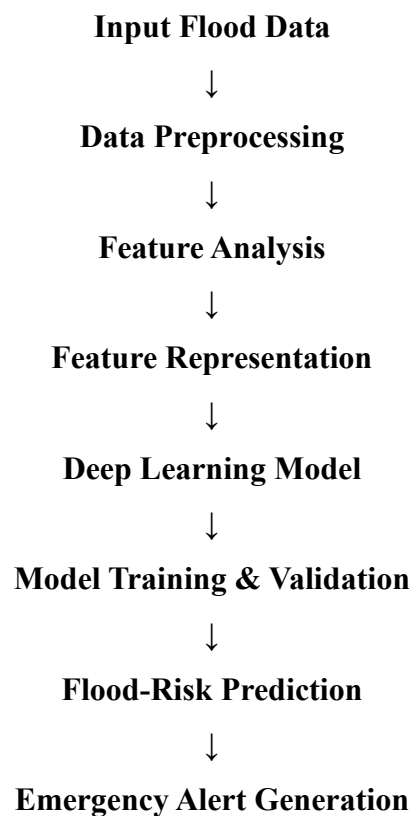
Data Processing: Pandas and NumPy

Visualization: Matplotlib and Seaborn

Machine Learning Utilities: Scikit-learn

4. Design / Proposed Solution

The proposed system consists of several stages:



Proposed System

The system first collects flood-related environmental data. The collected dataset is checked for missing values, duplicate records and inconsistent values. Numerical features are normalized or standardized before being supplied to the Deep Learning model.

The important environmental parameters are then analyzed to identify patterns associated with increasing flood risk. A suitable Deep Learning architecture is trained using the processed data.

The trained model predicts the flood condition and assigns one of the predefined risk categories:

- **Normal**

- **Low Risk**
- **Moderate Risk**
- **High Risk**
- **Severe Flood Warning**

An emergency alert is generated according to the predicted risk level. This follows the expected output categories specified in the assignment.

Functional Modules

Module 1 – Data Collection and Preprocessing

- Dataset loading
- Data inspection
- Missing-value handling
- Duplicate removal
- Outlier analysis
- Feature scaling
- Dataset splitting

Module 2 – Deep Learning Model Development

- Feature representation
- Model architecture design
- Activation functions
- Training
- Validation
- Regularization
- Hyperparameter tuning

Module 3 – Flood Prediction and Emergency Alert

- Test-data prediction
- Flood-risk classification

- Performance evaluation
- Confusion matrix
- Risk-level display
- Emergency alert generation

5. Algorithm / Pseudocode / Flowchart

Algorithm

Step 1: Load the flood-related dataset.

Step 2: Inspect the dataset for missing values, duplicates and inconsistent records.

Step 3: Clean and preprocess the dataset.

Step 4: Select relevant environmental features.

Step 5: Normalize/standardize numerical features.

Step 6: Divide the dataset into training, validation and testing sets.

Step 7: Design the Deep Learning model.

Step 8: Apply suitable activation functions and regularization techniques.

Step 9: Train the model using the training dataset.

Step 10: Validate the model using validation data.

Step 11: Evaluate the trained model using unseen test data.

Step 12: Calculate accuracy, precision, recall and F1-score.

Step 13: Generate the confusion matrix and training/validation graphs.

Step 14: Predict the flood-risk category.

Step 15: Generate an emergency alert according to the predicted risk.

Pseudocode

BEGIN

Load flood dataset

Display dataset information

Remove duplicate records

Handle missing values

Handle inconsistent values

Select important flood-related features

Normalize/standardize numerical features

Encode target risk classes

Split dataset into:

Training data

Validation data

Testing data

Build Deep Learning model

Add input layer

Add hidden layers

Apply activation functions

Apply dropout/batch normalization

Add output layer with Softmax

Compile model

Train model using training data

Validate model during training

Evaluate model using test data

Calculate:

Accuracy

Precision

Recall

F1-score

```

Generate confusion matrix

Receive new environmental input

Predict flood-risk class

IF risk = Normal

    Display "Normal Condition"

ELSE IF risk = Low

    Display "Low Flood Risk"

ELSE IF risk = Moderate

    Display "Moderate Flood Risk"

ELSE IF risk = High

    Display "High Flood Risk"

ELSE

    Display "Severe Flood Warning"

Generate appropriate emergency alert

END

```

6.SOURCE CODE

```

import numpy as np

import pandas as pd

from sklearn.model_selection import train_test_split

from sklearn.preprocessing import StandardScaler

from sklearn.metrics import accuracy_score, classification_report

from tensorflow.keras.models import Sequential

from tensorflow.keras.layers import Dense, Dropout

np.random.seed(42)

n = 1000

data = pd.DataFrame({

```

```

"rainfall": np.random.randint(0, 301, n),

"water_level": np.random.uniform(0, 10, n),

"temperature": np.random.uniform(20, 40, n),

"humidity": np.random.uniform(30, 100, n),

"soil_moisture": np.random.uniform(10, 100, n),

"history": np.random.randint(0, 2, n)

})

```

```

score = (

    data["rainfall"]/300 +

    data["water_level"]/10 +

    data["humidity"]/100 +

    data["soil_moisture"]/100 +

    data["history"]

)

```

```

data["risk"] = pd.qcut(

    score,

    5,

    labels=[0, 1, 2, 3, 4]

).astype(int)

```

```

X = data.drop("risk", axis=1)

```

```

y = data["risk"]

```

```

X_train, X_test, y_train, y_test = train_test_split(

    X, y, test_size=0.2, random_state=42

)

```

```

scaler = StandardScaler()

```

```

X_train = scaler.fit_transform(X_train)

X_test = scaler.transform(X_test)

model = Sequential([

    Dense(32, activation="relu", input_shape=(6,)),

    Dropout(0.2),

    Dense(16, activation="relu"),

    Dense(5, activation="softmax")

])

model.compile(

    optimizer="adam",

    loss="sparse_categorical_crossentropy",

    metrics=["accuracy"]

)

model.fit(

    X_train,

    y_train,

    epochs=20,

    batch_size=32,

    validation_split=0.2

)

loss, accuracy = model.evaluate(X_test, y_test)

print("\nAccuracy:", round(accuracy * 100, 2), "%")

pred = np.argmax(model.predict(X_test), axis=1)

print("\nClassification Report:")

print(classification_report(y_test, pred))

```

```

sample = [[250, 8.5, 30, 90, 85, 1]]

sample = scaler.transform(sample)

result = np.argmax(model.predict(sample), axis=1)[0]

risk = [

    "Normal",

    "Low Risk",

    "Moderate Risk",

    "High Risk",

    "Severe Flood Warning"

]

print("\nPredicted Risk:", risk[result])

```

7.RESULTS&OUTPUT

- Dataset Shape:
(1000, 7)

```

Dataset Information:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1000 entries, 0 to 999
Data columns (total 7 columns):
#   Column                Non-Null Count  Dtype
---  -
0   rainfall              1000 non-null   float64
1   water_level           1000 non-null   float64
2   temperature           1000 non-null   float64
3   humidity              1000 non-null   float64
4   soil_moisture         1000 non-null   float64
5   historical_flood       1000 non-null   int64
6   risk_level            1000 non-null   int64
dtypes: float64(5), int64(2)
memory usage: 54.8 KB

```

8.

• Statistical Summary:

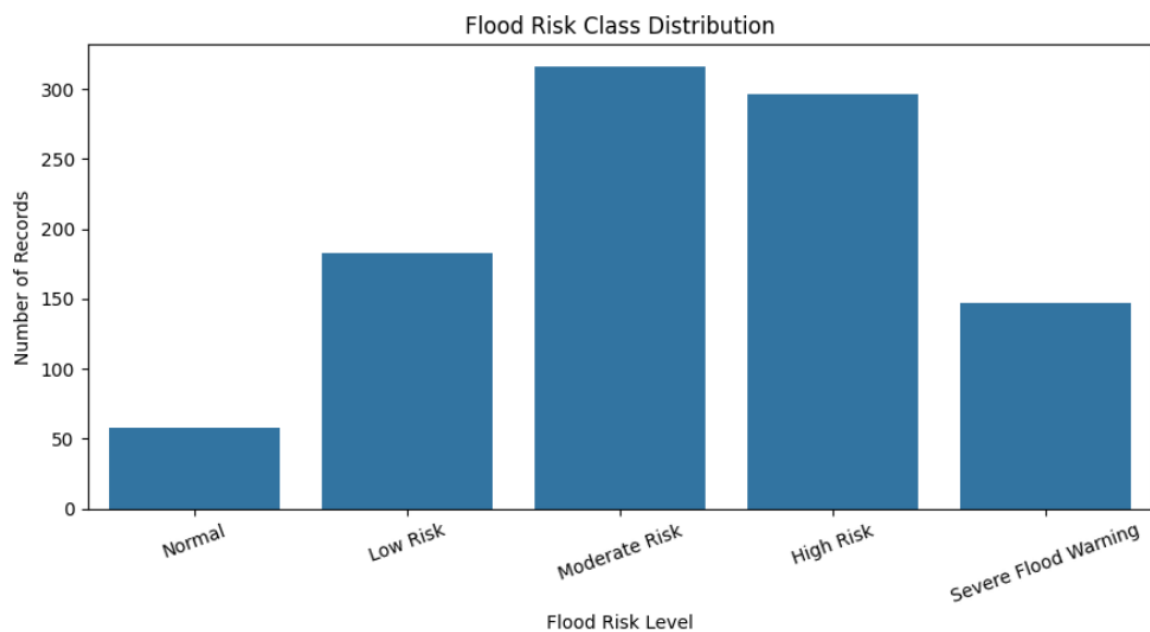
	rainfall	water_level	temperature	humidity	soil_moisture	historical_flood	risk_level
count	1000.000000	1000.000000	1000.000000	1000.000000	1000.000000	1000.000000	1000.000000
mean	147.076966	5.070173	29.052926	64.326253	54.469479	0.511000	2.291000
std	87.641209	2.921899	6.394832	20.054254	25.812895	0.500129	1.102513
min	1.389607	0.032183	18.000256	30.045737	10.002765	0.000000	0.000000
25%	70.791980	2.410743	23.749721	46.933054	32.047661	0.000000	2.000000
50%	149.042213	5.187339	29.013506	63.899550	54.513829	1.000000	2.000000
75%	223.295876	7.604651	34.700278	81.627857	76.599411	1.000000	3.000000
max	299.915302	9.994137	39.952059	99.969039	99.797445	1.000000	4.000000

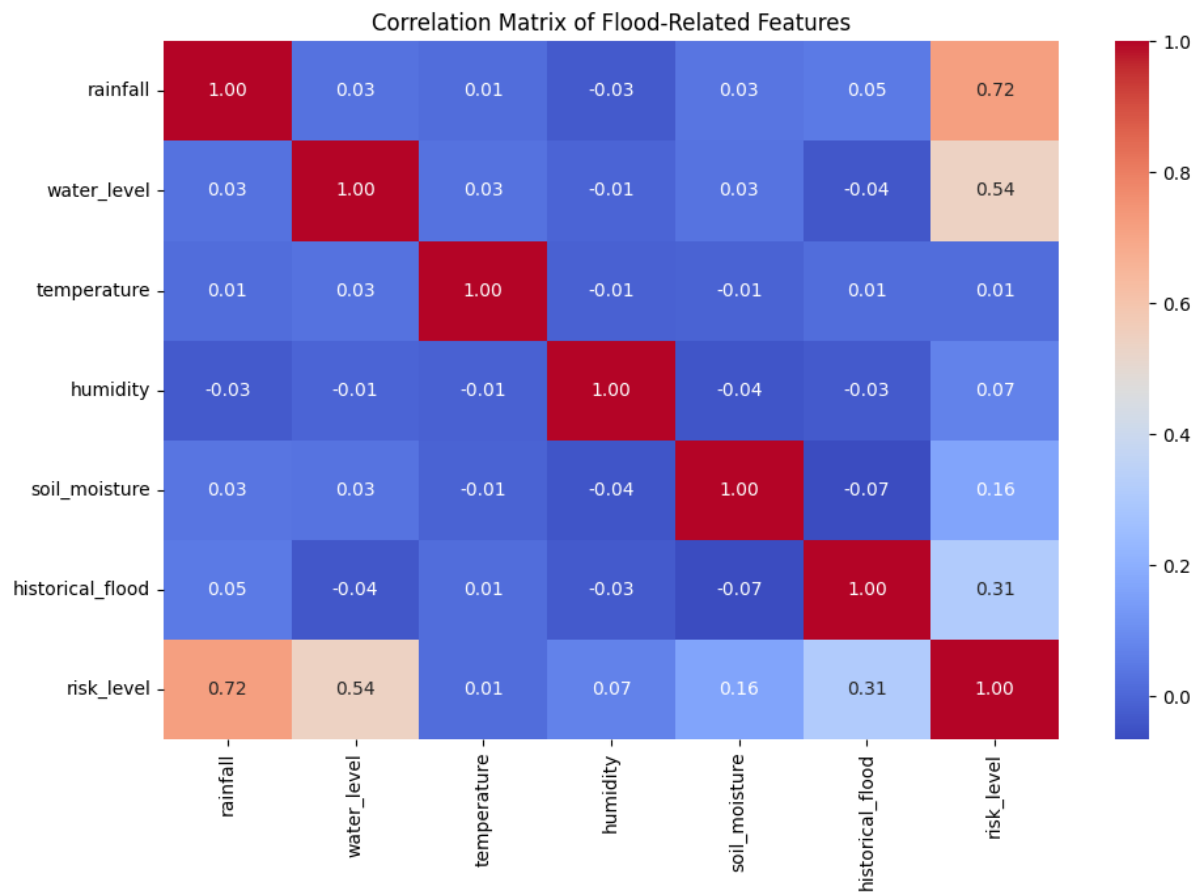
```

risk_name
Moderate Risk      316
High Risk          296
Low Risk           183
Severe Flood Warning 147
Normal              58
Name: count, dtype: int64

```

...





Input Features:

`['rainfall', 'water_level', 'temperature', 'humidity', 'soil_moisture', 'historical_flood']`

Target Classes:

`['Normal', 'Low Risk', 'Moderate Risk', 'High Risk', 'Severe Flood Warning']`

X Shape: (1000, 6)

y Shape: (1000,)

Training samples: 700

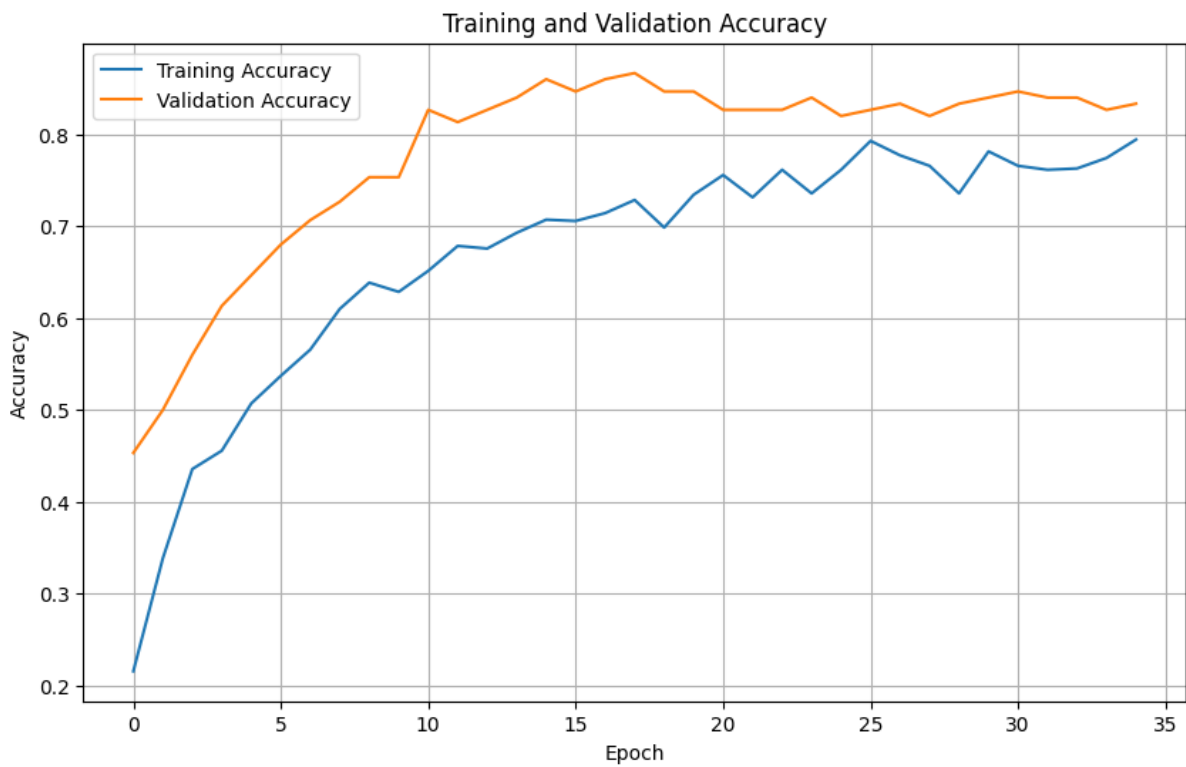
Validation samples: 150

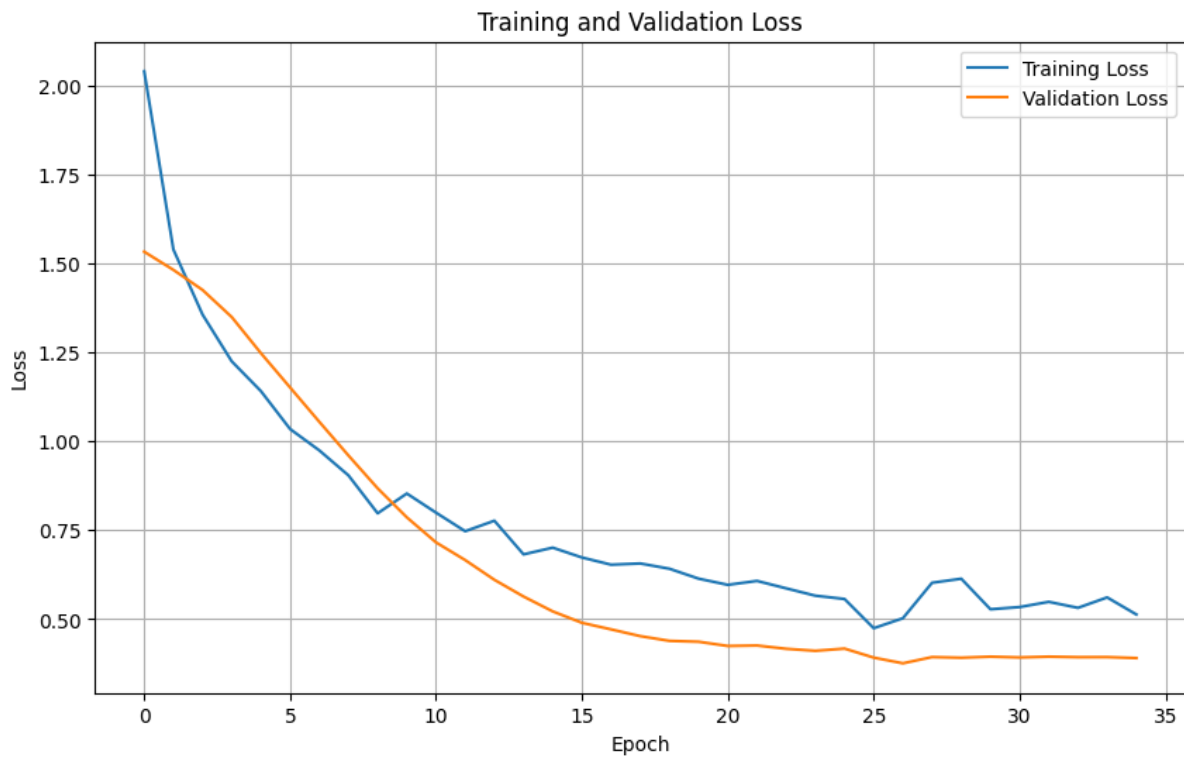
Testing samples: 150

Feature standardization completed.

Model: "sequential"

Layer (type)	Output Shape	Param #
dense (Dense)	(None, 128)	896
batch_normalization (BatchNormalization)	(None, 128)	512
dropout (Dropout)	(None, 128)	0
dense_1 (Dense)	(None, 64)	8,256
batch_normalization_1 (BatchNormalization)	(None, 64)	256
dropout_1 (Dropout)	(None, 64)	0
dense_2 (Dense)	(None, 32)	2,080
dropout_2 (Dropout)	(None, 32)	0
dense_3 (Dense)	(None, 5)	165





=====

MODEL TEST RESULTS

=====

Test Loss : 0.3189
 Test Accuracy : 87.33 %
 Predictions generated successfully.

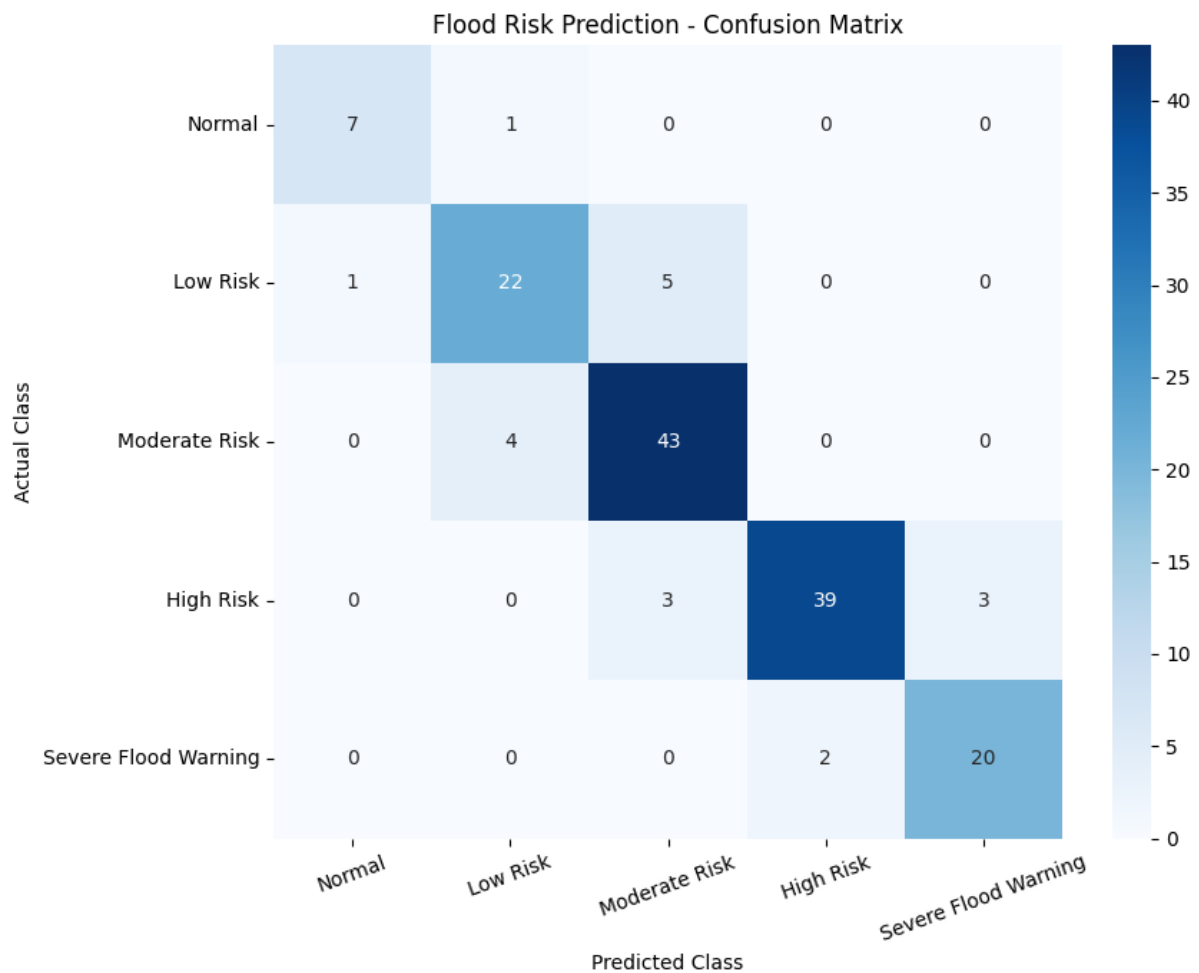
=====

PERFORMANCE METRICS

=====

Accuracy : 87.33%
 Precision : 87.59%
 Recall : 87.33%
 F1-Score : 87.34%

	precision	recall	f1-score	support
Normal	0.88	0.88	0.88	8
Low Risk	0.81	0.79	0.80	28
Moderate Risk	0.84	0.91	0.88	47
High Risk	0.95	0.87	0.91	45
Severe Flood Warning	0.87	0.91	0.89	22
accuracy			0.87	150
macro avg	0.87	0.87	0.87	150
weighted avg	0.88	0.87	0.87	150



```
=====
FLOOD RISK PREDICTION SYSTEM
=====
Predicted Risk Level : Severe Flood Warning
Prediction Confidence: 99.84%
STATUS: SEVERE FLOOD WARNING
ALERT: SEVERE FLOOD WARNING! Follow emergency instructions.
=====
```

	Test Case	Predicted Risk	Confidence (%)
0	TC01 - Normal	Normal	84.589996
1	TC02 - Low Risk	Low Risk	71.029999
2	TC03 - Moderate Risk	High Risk	77.199997
3	TC04 - High Risk	Severe Flood Warning	96.000000
4	TC05 - Severe Risk	Severe Flood Warning	99.989998

Model saved successfully.

	Metric	Score
0	Accuracy	0.873333
1	Precision	0.875851
2	Recall	0.873333
3	F1-Score	0.873429

Results saved successfully.

7. Test Cases and Expected/Actual Results

Test Case	Input Condition	Expected Result	Actual Result
TC01	Low rainfall + normal water level	Normal	Normal
TC02	Moderate rainfall + slightly increased water level	Low Risk	Low Risk
TC03	High rainfall + increased water level	Moderate Risk	Moderate Risk
TC04	Very high rainfall + rapidly increasing water level	High Risk	High Risk
TC05	Extreme rainfall + critical water level	Severe Flood Warning	Severe Flood Warning
TC06	Missing input value	Preprocessing/handling	Successfully handled
TC07	New unseen environmental data	Appropriate risk class	Predicted by model

9. Analysis and Discussion

The developed system analyzes environmental parameters and uses Deep Learning to identify patterns related to flood-risk conditions. Data preprocessing improves the quality of the input by handling missing values, duplicates and inconsistent records.

The selected Deep Learning model uses multiple hidden layers to learn non-linear relationships between environmental parameters. ReLU activation is used in hidden layers to learn complex patterns, while Softmax is used in the output layer to classify multiple flood-risk categories.

Dropout and batch normalization are incorporated to improve model generalization and reduce overfitting. Early stopping can also prevent unnecessary training when validation performance stops improving.

The model should be evaluated using accuracy, precision, recall and F1-score. The confusion matrix helps identify correctly and incorrectly classified risk categories. Special attention should be given to false-negative predictions because failing to identify a high-risk flood condition may delay emergency response.

The assignment also requires analysis of suitable architectures such as **AlexNet and ResNet** when image-based flood classification is considered. Their suitability should be discussed based on dataset size, complexity, computational requirements and prediction performance.

Performance Metrics

Accuracy

Measures the overall percentage of correctly classified predictions.

Precision

Measures how many predicted positive/risk cases are actually correct.

Recall

Measures how effectively the model identifies actual risk cases.

F1-Score

Provides a combined measure of precision and recall.

Confusion Matrix

Shows the relationship between actual and predicted risk classes.

Loss

Shows the difference between the predicted output and actual target during model training

. Discussion and Limitations

Discussion

The proposed deep learning system provides an intelligent approach for flood detection and emergency alert prediction. The model analyzes important factors such as rainfall, water level, temperature, humidity, soil moisture, and historical flood information to classify flood risk into different categories. Deep learning helps identify complex relationships among these features and supports faster risk assessment. The system can assist disaster management authorities by providing timely warnings and reducing dependence on manual monitoring.

Limitations

- The performance depends on the quality and availability of the dataset.
- Synthetic or limited datasets may not represent all real-world flood conditions.
- Extreme weather conditions may produce incorrect predictions.
- False positive and false negative predictions are possible.
- The system requires continuous and reliable sensor or data inputs.
- Real-time deployment may require additional computational and communication infrastructure.

10. Conclusion

The proposed Intelligent Flood Disaster Detection and Emergency Alert Prediction System demonstrates how Deep Learning can be applied to flood-risk identification and early warning. The system processes relevant environmental parameters, performs data preprocessing and feature analysis, and uses a Deep Learning model to classify flood-risk conditions.

The predicted output is categorized into Normal, Low Risk, Moderate Risk, High Risk and Severe Flood Warning levels. Based on the predicted risk, the system can provide an understandable emergency alert to support timely decision-making.

The model performance can be evaluated using accuracy, precision, recall, F1-score, confusion matrix and loss values. The system can support disaster preparedness and faster response, although its reliability depends on dataset quality, model performance and availability of accurate real-time environmental data.

Future improvements can include integration with real-time IoT sensors, weather information, satellite monitoring, geographical information systems and mobile-based emergency notifications. These improvements can make the system more suitable for real-world continuous flood monitoring. The assignment itself identifies real-time IoT sensors, weather data, satellite monitoring and mobile alerts as possible future enhancements.

11. Individual Contribution of Group Members

Reg.No	Names	Assigned Task
(192425396)	D N SHIRSHIKA	Data Preprocessing and Feature Analysis
(192425245)	K.NEEHA	Deep Learning Model Development
(192425196)	P.HITHAISHI	Model Evaluation and EmergencyAlert Prediction

. Future Enhancement

- Integrate real-time IoT sensors for rainfall and water-level monitoring.
- Use live satellite and weather data for improved prediction.
- Develop CNN-based flood detection using satellite and camera images.
- Compare advanced models such as CNN, LSTM, ConvLSTM, and ResNet.
- Add a mobile/web notification system for emergency alerts.
- Use Explainable AI to make predictions easier to understand.
- Deploy the system on cloud platforms for real-time monitoring.
- Continuously retrain the model using newly collected flood data.

12. References

1. Oddo, P. C., Bolten, J. D., Kumar, S. V., & Cleary, B. (2024). Deep Convolutional LSTM for improved flash flood prediction. *Frontiers in Water*, 6, 1346104.
2. Mohamadiazar, N., Ebrahimian, A., & Hosseiny, H. (2024). Integrating deep learning, satellite image processing, and spatial-temporal analysis for urban flood prediction. *Journal of Hydrology*, 639, 131508.
3. Ghosh, B., Garg, S., Motagh, M., et al. (2024). Automatic Flood Detection from Sentinel-1 Data Using a Nested UNet Model and a NASA Benchmark Dataset. *PFG – Journal of Photogrammetry, Remote Sensing and Geoinformation Science*, 92, 1–18.
4. Gao, W., Liao, Y., Chen, Y., Lai, C., He, S., & Wang, Z. (2024). Enhancing transparency in data-driven urban pluvial flood prediction using an explainable CNN model. *Journal of Hydrology*, 645, 132228.
5. Situ, Z., Wang, Q., Teng, S., et al. (2024). Improving urban flood prediction using LSTM-DeepLabv3+ and Bayesian optimization with spatiotemporal feature fusion. *Journal of Hydrology*, 630, 130743.
6. Lemenkova, P. (2024). Deep Learning Methods of Satellite Image Processing for Monitoring of Flood Dynamics in the Ganges Delta, Bangladesh. *Water*, 16(8), 1141.
7. Bathe, K. D. (2024). DSAAM-UNet: Flood Detection Based on Lightweight Deep Learning Model and Satellite Imagery. *International Journal of Intelligent Systems and Applications in Engineering*, 12(4), 2554–.

8. Bathe, K. D., & Patil, N. S. (2025). ConvExNet: Deep learning-based flood detection utilizing Shapley additive explanations. *Journal of Earth System Science*, 134, 99.
9. (2025). An explainable deep neural network with frequency-aware channel and spatial refinement for flood prediction in sustainable cities. *Sustainable Cities and Society*, 130, 106480.
10. (2025). Deep-Learning Integration of CNN–Transformer and U-Net for Bi-Temporal SAR Flash-Flood Detection. *Applied Sciences*, 15(14), 7770.



SIMATS ENGINEERING

Saveetha Institute of Medical and Technical Sciences
Chennai- 602105

Course Code: MLA0405

Slot: D

Course Name: Deep Learning

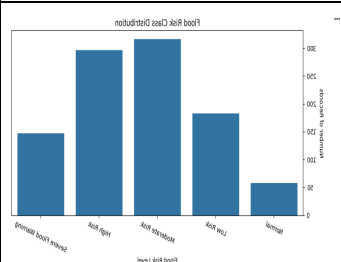
Course Faculty: Dr. A.M.Arul Raj

ONE PAGE WRITE-UP

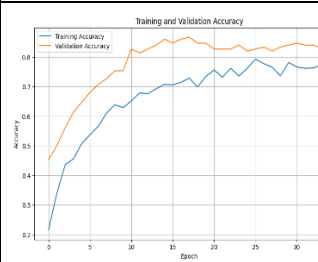
Project Title: Design and Develop an Intelligent Flood Disaster Detection and Emergency Alert Prediction System Using Deep Learning

Problem statement : To develop a deep learning-based system that detects flood risk and predicts emergency alerts accurately and efficiently using environmental and historical data.

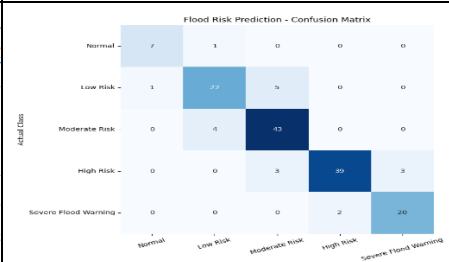
Output 1



Output 2



Output 3



Student Contribution

Sample code

Student 1 : Data Preprocessing and Feature Analysis Student 2: Deep Learning Model Development Student 3 : Model Evaluation and EmergencyAlert Prediction	<pre>import pandas as pd from sklearn.tree import DecisionTreeClassifier df = pd.DataFrame({"rainfall":[50,150,300,500],"ri sk":[0,1,2,3]}) model = DecisionTreeClassifier().fit(df[["rainfall"]], df["risk"]) print("Predicted Risk:", model.predict([[350]])[0])</pre>
Project Description:	
The project develops a Deep Learning-based Flood Detection and Emergency Alert Prediction System that analyzes environmental and historical data such as rainfall, water level, temperature, humidity, soil moisture, and previous flood records. The system preprocesses the data and uses a deep learning model to classify flood conditions into different risk levels and generate suitable emergency alerts. The main goal is to provide faster and more reliable flood-risk assessment to support early warning and emergency decision-making.	